ytLending理论

totalSupplyBase/totalBorrowBase/totalSupply/totalBorrow含义解释

totalSupplyBase和totalBorrowBase分别代表所有用户的本金（正负principal）之和

代码块

```
// T0 时刻：用户A存入 1000 USDC
accrueInterest()  // supplyIndex = 1.0
oldBalance = 0
newBalance = 0 + 1000 = 1000
newPrincipal = 1000 / 1.0 = 1000
totalSupplyBase += 1000  // totalSupplyBase = 1000

// T1 时刻：10天后，什么都不做
accrueInterest()  // supplyIndex = 1.05 (增长了5%)
// 注意：totalSupplyBase 仍然是 1000（不变）
// 但实际余额 = 1000 * 1.05 = 1050

// T2 时刻：用户A再存入 500 USDC
accrueInterest()  // supplyIndex = 1.05
oldPrincipal = 1000
oldBalance = 1000 * 1.05 = 1050  // 先计算出含利息的余额
newBalance = 1050 + 500 = 1550
newPrincipal = 1550 / 1.05 ≈ 1476.19  // 用最新 index 算出新进用户的principal计入本金，因为新用户使用的index是增长后的index，所以principal不是500，而是476.19

// 更新全局状态
supplyAmount = newPrincipal - oldPrincipal = 1476.19 - 1000 = 476.19
totalSupplyBase += 476.19  // totalSupplyBase = 1476.19
```

储备金的概念

储备金随着大量的存取的交易，会越来越多。这是协议的收入，同时也要承担坏帐的出现。

储备金 = balance - totalSupply + totalBorrow（totalSuply等于银行欠用户，totalBorrow等于用户欠银行）

代码块

```
场景 1：初始状态（空合约）
```

```
实际资金 = 0
totalSupply = 0 (没人存款)
totalBorrow = 0 (没人借款)
储备金 = 0 + 0 - 0 = 0

场景 2: 用户A存入 1000 USDC
实际资金 = 1000 (收到现金)
totalSupply = 1000 (欠用户A的钱)
totalBorrow = 0
储备金 = 1000 - 1000 + 0 = 0
解释: 收到1000, 但也欠用户1000, 所以没有储备金

场景 3: 用户B借走 600 USDC
实际资金 = 1000 - 600 = 400 (支出现金)
totalSupply = 1000 (仍欠用户A)
totalBorrow = 600 (用户B欠合约)
储备金 = 400 - 1000 + 600 = 0
解释:
资产: 400现金 + 600应收 = 1000
负债: 1000应付
储备金: 1000 - 1000 = 0

场景 4: 10天后, 累积利息
// 假设:
// - 用户A的存款利息 = 10 USDC
// - 用户B的借款利息 = 30 USDC
实际资金 = 400 (不变, 利息还没实际支付)
totalSupply = 1010 (欠用户A 1010, 含利息)
totalBorrow = 630 (用户B欠 630, 含利息)
储备金 = 400 - 1010 + 630 = 20

解释:
借款利息(30) - 存款利息(10) = 20, 这是协议赚的利差!
这个利差就是储备金
```

清算过程

代码块

```
阶段 1: 清算 (absorb)
// ========== 清算前 ==========
balance = 10000 USDC
totalSupply = 8000
totalBorrow = 5000 (包含被清算用户的1500)
```

```
reserves = 10000 - 8000 + 5000 = 7000
// ========== 执行 absorb ==========
用户债务: 1500 USDC
用户抵押品: 2 ETH (之前价值 2000 USDC, 现在价值 1200 USDC)
坏账: 300 USDC (未及时清算导致坏帐出现)
// 清算操作:
userCollateral[borrower][ETH] = 0  // 抵押品转入协议, 用户抵押品记录清零
collateralReserves[ETH] += 2 ETH   // 抵押品记录存入清算库存
totalBorrowBase -= 1500            // 由于用户债务被清算, 债务记录清除
// ========== 清算后 (未售出抵押品前) ==========
balance = 10000 USDC  // 没有增加! 只是收到了 ETH, 不是 USDC
totalSupply = 8000 (如果抵押品不够)
totalBorrow = 3500 (5000 - 1500) // 用户总借款减少了 1500
reserves = 10000 - 8000 + 3500 = 5500  // 储备金减少了 1500!
问题来了: totalBorrow 减少了 1500, 但 balance 没增加, 所以 reserves 减少了 1500!

阶段 2: 售出抵押品 (buyCollateral)
// ========== buyCollateral 前 ==========
balance = 10000 USDC
totalSupply = 8000
totalBorrow = 3500
reserves = 5500
collateralReserves[ETH] = 2 ETH
// ========== 执行 buyCollateral ==========
// 清算人用 1200 USDC 购买 2 ETH (打折出售)
IERC20(baseToken).safeTransferFrom(msg.sender, address(this), 1200);
collateralReserves[ETH] -= 2 ETH;
IERC20(WETH).safeTransfer(recipient, 2 WETH); (转移的是ETH对应的ERC20资产)
// ========== buyCollateral 后 ==========
balance = 10000 + 1200 = 11200 USDC  // 增加了 1200
totalSupply = 8000  // 不变
totalBorrow = 3500  // 不变 (在 absorb 时已经减少了)
reserves = 11200 - 8000 + 3500 = 6700  // 增加了 1200

完整对比: 清算前 vs 最终
// 清算前:
reserves = 7000
// 清算 + 售出抵押品后:
reserves = 6700
// 净变化: -300 (这就是坏账! )
```

利率参数含义

```
1. 供应利率（Supply Rate）- 存款利率
uint64 constant SUPPLY_KINK = 0.8e18;          // 80%
uint64 constant SUPPLY_RATE_LOW = 0.03e18;     // 3% APY
uint64 constant SUPPLY_RATE_HIGH = 0.4e18;     // 40% APY
uint64 constant SUPPLY_RATE_BASE = 0;          // 设置为0%，当没有借款时，存款人不获得利息
含义：
SUPPLY_KINK = 80%（拐点）
当资金利用率 < 80% 时，使用低斜率计算利率
当资金利用率 > 80% 时，使用高斜率计算利率
这是利率曲线的转折点
SUPPLY_RATE_LOW = 3%（低斜率）
在利用率 0-80% 区间，每 1% 利用率增加，利率增加约 0.0375%
计算：3% / 80 = 0.0375%
SUPPLY_RATE_HIGH = 40%（高斜率）
在利用率 80-100% 区间，每 1% 利用率增加，利率增加约 2%
计算：40% / 20 = 2%
高斜率激励存款人在高利用率时存款
SUPPLY_RATE_BASE = 0（基础利率）
即使利用率为 0，也有这个基础利率（这里是 0）
利率计算公式：
if (utilization <= SUPPLY_KINK) {
    rate = SUPPLY_RATE_BASE + (utilization * SUPPLY_RATE_LOW / SUPPLY_KINK)
} else {
    rate = SUPPLY_RATE_BASE + SUPPLY_RATE_LOW +
           ((utilization - SUPPLY_KINK) * SUPPLY_RATE_HIGH / (1 - SUPPLY_KINK))
}
实际例子：
利用率 = 50%:  rate ≈ 0 + (0.5 * 3% / 0.8) = 1.875%
利用率 = 80%:  rate ≈ 3%
利用率 = 90%:  rate ≈ 3% + (0.1 * 40% / 0.2) = 23%
利用率 = 100%: rate ≈ 3% + 40% = 43%

2. 借款利率（Borrow Rate）- 贷款利率
uint64 constant BORROW_KINK = 0.8e18;          // 80%
uint64 constant BORROW_RATE_LOW = 0.05e18;     // 5% APY
uint64 constant BORROW_RATE_HIGH = 1.5e18;     // 150% APY
uint64 constant BORROW_RATE_BASE = 0.015e18;   // 1.5% base
含义：
BORROW_KINK = 80%（拐点）
和供应利率一样，80% 是转折点
BORROW_RATE_LOW = 5%（低斜率）
在利用率 0-80% 区间的斜率
每 1% 利用率增加，利率增加 5% / 80 = 0.0625%
BORROW_RATE_HIGH = 150%（高斜率）
在利用率 80-100% 区间的斜率
每 1% 利用率增加，利率增加 150% / 20 = 7.5%
```

```
非常陡峭！用来惩罚高利用率时的借款
BORROW_RATE_BASE = 1.5%（基础利率）
即使利用率为 0，借款人也要支付 1.5% 的基础利率
实际例子：
利用率 = 0%:   rate = 1.5% (base)
利用率 = 50%:  rate ≈ 1.5% + (0.5 * 5% / 0.8) = 4.625%
利用率 = 80%:  rate ≈ 1.5% + 5% = 6.5%
利用率 = 90%:  rate ≈ 6.5% + (0.1 * 150% / 0.2) = 81.5%
利用率 = 100%: rate ≈ 1.5% + 5% + 150% = 156.5%
利率曲线可视化
借款利率 (Borrow Rate)
156.5%|                                /
      |                              /
      |                            /
 81.5%|                         /
      |                       /
  6.5%|____________拐点___/
      |            /
  4.6%|         /
  1.5%|_____/_________________
       0%    50%    80%  90% 100%  利用率
存款利率 (Supply Rate)
  43%|                             /
     |                           /
  23%|                         /
     |                       /
   3%|___________拐点__/
     |          /
 1.9%|       /
   0%|____/_____________________
      0%   50%   80%  90%  100%  利用率
抵押品参数 (Collateral Parameters)

3. 抵押品系数
uint64 constant BORROW_CF = 0.80e18;              // 80% LTV
uint64 constant LIQUIDATE_CF = 0.85e18;           // 85% 清算线
uint64 constant LIQUIDATION_FACTOR = 0.95e18;     // 95% 清算系数
uint64 constant STORE_FRONT_PRICE_FACTOR = 0.5e18; // 50% 折扣系数
含义：
BORROW_CF = 80% (Borrow Collateral Factor / LTV)
作用：决定用户能借多少钱
含义：1 美元的抵押品，可以借 0.8 美元
例子：
用户存入：10 ETH × $2000 = $20,000
借款能力：$20,000 × 80% = $16,000
LIQUIDATE_CF = 85% (Liquidate Collateral Factor)
作用：决定何时触发清算
```

```
含义：当债务价值 > 抵押品价值 × 85% 时，可以被清算
例子：
抵押品：10 ETH × $2000 = $20,000
清算阈值：$20,000 × 85% = $17,000
当前借款：$16,000
安全状态：$16,000 < $17,000
如果 ETH 价格跌到 $1700：
抵押品：10 ETH × $1700 = $17,000
清算阈值：$17,000 × 85% = $14,450
当前借款：$16,000
危险状态：$16,000 > $14,450 可以被清算！
安全缓冲区
BORROW_CF (80%) ─┬─ 借款上限
                 │
                 │ 5% 安全缓冲
                 │
LIQUIDATE_CF (85%) ─ 清算线
LIQUIDATION_FACTOR = 95% (清算折扣)
作用：清算时抵押品的折价系数
含义：清算时，抵押品按 95% 的价值计算
为什么要折扣？
市场波动：价格在清算过程中可能变化
Gas 成本：清算人需要支付 Gas
风险补偿：激励清算人快速行动
例子：
用户债务：$16,000
抵押品实际价值：10 ETH × $1700 = $17,000
清算时抵押品价值：$17,000 × 95% = $16,150
协议收到：$16,150 的抵押品
还掉债务：$16,000
剩余：$150 (给用户)
STORE_FRONT_PRICE_FACTOR = 50% (购买折扣)
作用：清算人购买抵押品时的额外折扣
含义：在 liquidationFactor 基础上，再打 50% 折扣
计算公式：
discountFactor = STORE_FRONT_PRICE_FACTOR × (1 - LIQUIDATION_FACTOR)
               = 50% × (1 - 95%)
               = 50% × 5%
               = 2.5%
最终购买价格 = 市场价 × (1 - 2.5%) = 市场价 × 97.5%
例子：
// 购买 10 ETH 需要支付
清算人支付 = 10 ETH × $1657.5 = $16,575
// ==================== 清算人的利润计算 ====================
清算人支付：$16,575 USDC
清算人获得：10 ETH (市场价 $17,000)
清算人利润：$17,000 - $16,575 = $425
```

```
// ==================== 协议的账本 ====================
收入：$16,575（清算人支付）
支出：$16,000（偿还存款人，清零用户债务）
协议储备金增加：$16,575 - $16,000 = $575
```